ARQUITECTURA DE SOFTWARE

SISTEMA DE ATENCIÓN CIUDADANA (SAC) - MUNICIPALIDAD DE SAN AGUSTÍN DE CAJAS

1. INTRODUCCIÓN Y PROPÓSITO

El presente documento describe la arquitectura de software desarrollada para el **Sistema de Atención Ciudadana (SAC)** de la Municipalidad de San Agustín de Cajas. El diseño adopta el paradigma de **Programación Orientada a Objetos (POO)** bajo la arquitectura en capas **Modelo-Vista-Controlador (MVC)** y el **Patrón Repositorio (Repository Pattern)**.

Esta arquitectura garantiza una separación clara de responsabilidades, desacoplando la interfaz de usuario, la lógica de negocio municipal y la capa de persistencia en la base de datos NoSQL MongoDB Atlas.

2. ESTRUCTURA DE PAQUETES Y COMPONENTES (POO)

La organización modular del código fuente Java sigue el estándar de diseño orientado a objetos:

```
pe.gob.sac/
├── modelo/                    # Entidades del Dominio e Interfaces
│   ├── Solicitud.java         # Objeto de dominio principal
│   ├── Usuario.java           # Entidad de usuario y roles
│   └── SolicitudRepository.java # Interfaz de persistencia
├── dao/                       # Capa de Acceso a Datos (Implementación)
│   ├── SolicitudRepositoryMongo.java # Persistencia concreta en MongoDB
│   └── MongoUtil.java         # Configuración de conexión Singleton
├── controlador/               # Capa de Lógica de Negocio
│   └── SolicitudController.java # Validación y orquestación
└── vista/                     # Capa de Presentación
    ├── SolicitudServlet.java  # Controlador de peticiones HTTP
    └── *.jsp                  # Interfaces de usuario dinámicas
```

3. DEFINICIÓN DE COMPONENTES Y RESPONSABILIDADES

A. Entidad del Dominio (Solicitud.java)

Representa la entidad central del sistema. Encapsula los datos de los trámites y reclamos de los ciudadanos.

Atributo	Tipo	Descripción
codigo	String	Identificador único del trámite (ej. SAC-2026-000001).
tipo	String	Categoría de atención (Trámite,

Atributo	Tipo	Descripción
		Consulta, Pago, Reclamo).
descripcion	String	Detalle del requerimiento ciudadana.
estado	String	Estado actual de la solicitud (Registrada, En Proceso, Resuelta).
areaActual	String	Área municipal responsable de la atención.
fechaRegistro	Date	Timestamp automático del registro.

B. Interfaz Repositorio (SolicitudRepository.java)

Define el contrato de operaciones de persistencia sin acoplar el código a un motor de base de datos específico:

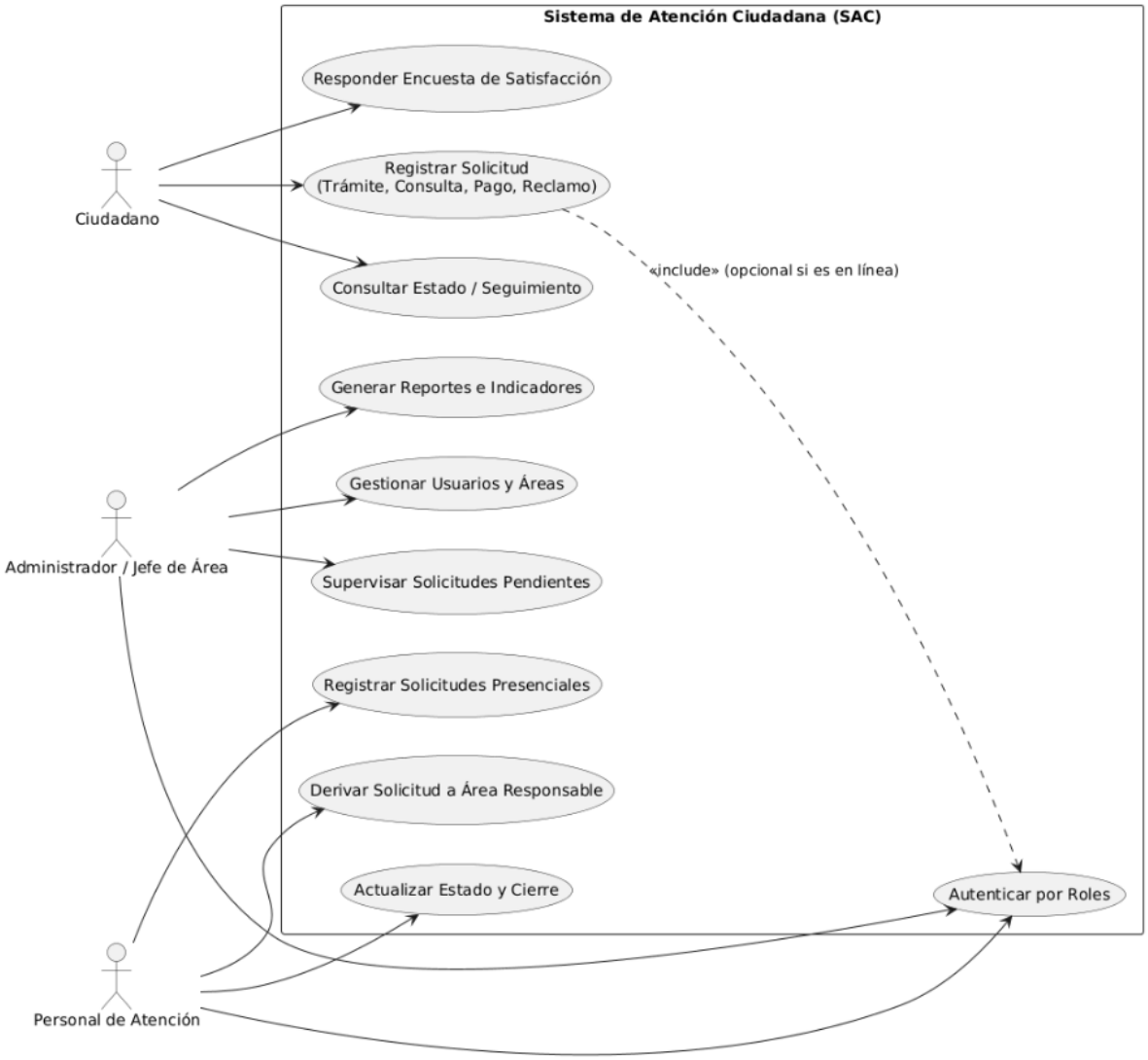
- void guardar(Solicitud solicitud)
- Solicitud buscarPorCodigo(String codigo)
- List<Solicitud> listarTodas()
- boolean existeCodigo(String codigo)

C. Controlador de Negocio (SolicitudController.java)

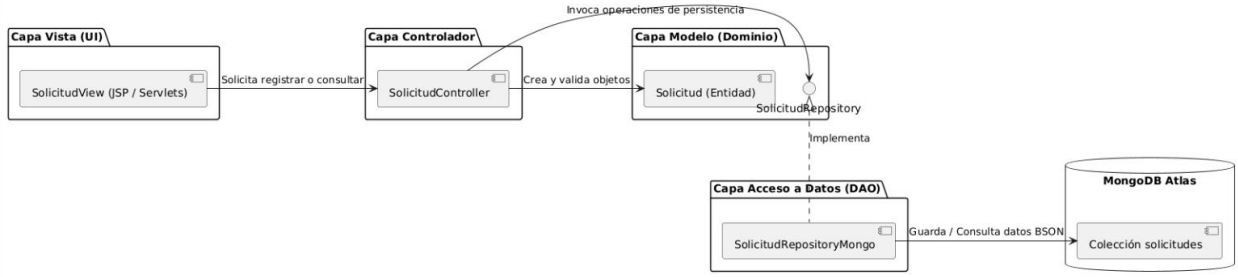
Coordina el flujo entre la vista y el repositorio. Aplica las validaciones de negocio (verificación de duplicados, datos completos) antes de procesar cualquier acción.

4. DIAGRAMAS ARQUITECTÓNICOS

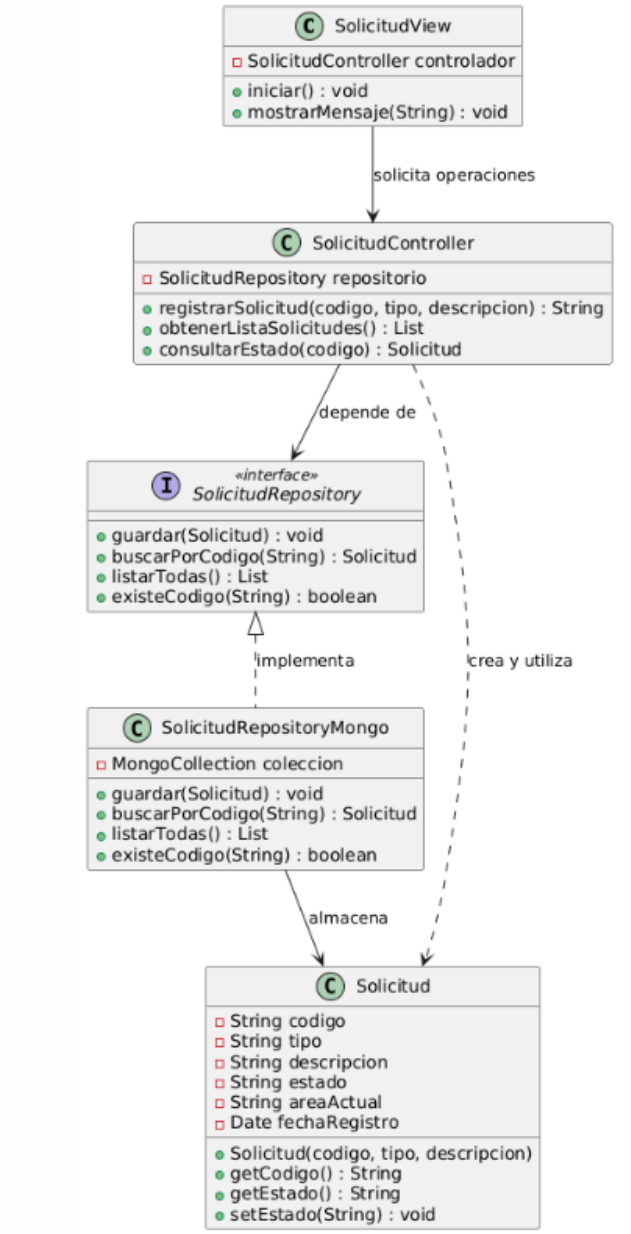
A . Diagrama de caso de uso


Sistema de Atención Ciudadana (SAC)
Responder Encuesta de Satisfacción
Registrar Solicitud
(Trámite, Consulta, Pago, Reclamo)
Consultar Estado / Seguimiento
«include» (opcional si es en línea)
Generar Reportes e Indicadores
Gestionar Usuarios y Áreas
Supervisar Solicitudes Pendientes
Registrar Solicitudes Presenciales
Derivar Solicitud a Área Responsable
Actualizar Estado y Cierre
Autenticar por Roles
Ciudadano
Administrador / Jefe de Área
Personal de Atención

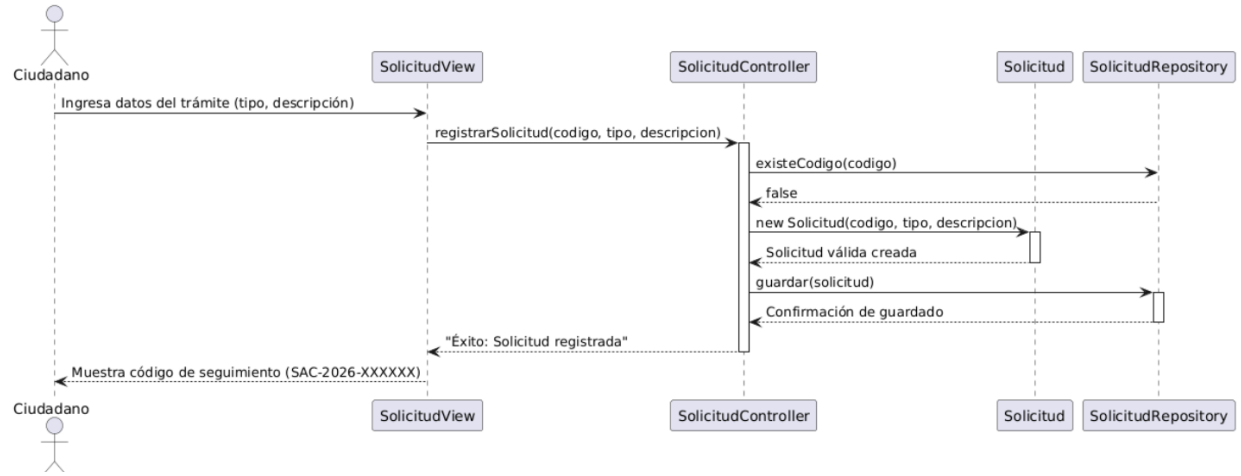
B. Diagrama Arquitectónico (Estructura de Capas)



C. Diagrama de Clases Simplificado (POO)



C. Diagrama de Secuencia (Flujo de Ejecución del Registro)



---

5. DESCRIPCIÓN DEL FLUJO DE EJECUCIÓN

El proceso de registro de una solicitud ciudadana sigue la siguiente secuencia analítica:

1. **Interacción Inicial:** El ciudadano ingresa los datos del trámite (tipo y descripción) en la interfaz web (SolicitudView).
2. **Recepción:** La vista captura la petición HTTP y transfiere la información al SolicitudController.
3. **Validación:** El controlador verifica que el código asignado sea único mediante el método existeCodigo() del repositorio.
4. **Instanciación:** Se instancia un nuevo objeto Solicitud. El constructor del modelo valida la consistencia interna de los datos.
5. **Persistencia:** El controlador invoca el método guardar(solicitud) a través de la interfaz SolicitudRepository.
6. **Almacenamiento:** La clase SolicitudRepositoryMongo transforma la entidad en un documento BSON y lo registra en la colección solicitudes de MongoDB Atlas.
7. **Respuesta:** El controlador retorna una confirmación de éxito a la vista, la cual despliega el código de seguimiento correspondiente al ciudadano.